פרויקט סוף מעבדה מערכות ספרתיות :

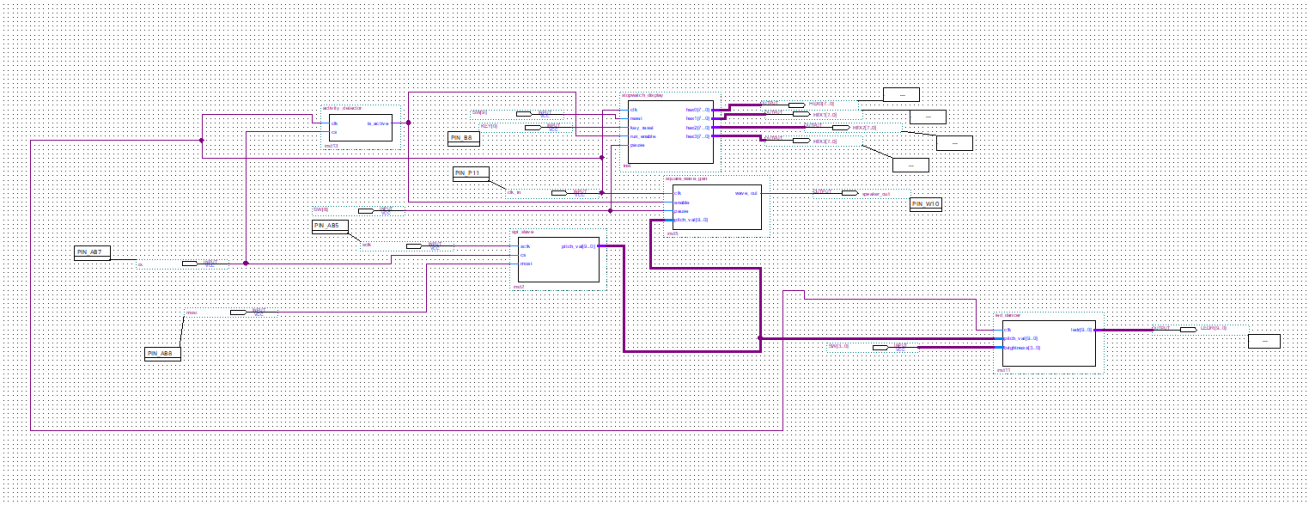
שם הפרויקט :סינתיסייזר מוזיקלי מבוסס SPI ומערכת בקרה מסונכרנת.

תיאור הפרויקט :

הפרויקט מציג מערכת משובצת המשלבת חומרה (FPGA DE10-Lite) ותוכנה (Raspberry Pi 5). המטרה היא יצירת סינתיסייזר מוזיקלי המופעל על ידי פקודות דיגיטליות ב-SPI, תוך סנכרון בזמן אמת עם שעון עצר (Stopwatch) ותצוגת לדים דינמית (LED Dancer) , סוייץ של עצירת המוזיקה והשעון וכפתור איפוס של השעון.

צילומי מסך :

שרטוט BDF :



הגדרת פינים והתאמת מתחים :

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation	Bus Hold	Weak Pull-Up Resist
in clk_in	Input	PIN_P11	3	B3_N0	PIN_P11	3.3-V LVTTL		8mA (default)					
in cs	Input	PIN_AB7	3	B3_N0	PIN_AB7	3.3-V LVTTL		8mA (default)					on
out HEX0[7]	Output	PIN_D15	7	B7_N0	PIN_D15	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX0[6]	Output	PIN_C17	7	B7_N0	PIN_C17	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX0[5]	Output	PIN_D17	7	B7_N0	PIN_D17	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX0[4]	Output	PIN_E16	7	B7_N0	PIN_E16	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX0[3]	Output	PIN_C16	7	B7_N0	PIN_C16	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX0[2]	Output	PIN_C15	7	B7_N0	PIN_C15	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX0[1]	Output	PIN_E15	7	B7_N0	PIN_E15	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX0[0]	Output	PIN_C14	7	B7_N0	PIN_C14	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[7]	Output	PIN_A16	7	B7_N0	PIN_A16	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[6]	Output	PIN_B17	7	B7_N0	PIN_B17	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[5]	Output	PIN_A18	7	B7_N0	PIN_A18	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[4]	Output	PIN_A17	7	B7_N0	PIN_A17	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[3]	Output	PIN_B16	7	B7_N0	PIN_B16	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[2]	Output	PIN_E18	6	B6_N0	PIN_E18	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[1]	Output	PIN_D18	6	B6_N0	PIN_D18	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX1[0]	Output	PIN_C18	7	B7_N0	PIN_C18	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[7]	Output	PIN_A19	7	B7_N0	PIN_A19	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[6]	Output	PIN_B22	6	B6_N0	PIN_B22	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[5]	Output	PIN_C22	6	B6_N0	PIN_C22	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[4]	Output	PIN_B21	6	B6_N0	PIN_B21	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[3]	Output	PIN_A21	6	B6_N0	PIN_A21	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[2]	Output	PIN_B19	7	B7_N0	PIN_B19	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[1]	Output	PIN_A20	7	B7_N0	PIN_A20	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX2[0]	Output	PIN_B20	6	B6_N0	PIN_B20	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[7]	Output	PIN_D22	6	B6_N0	PIN_D22	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[6]	Output	PIN_E17	6	B6_N0	PIN_E17	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[5]	Output	PIN_D19	6	B6_N0	PIN_D19	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[4]	Output	PIN_C20	6	B6_N0	PIN_C20	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[3]	Output	PIN_C19	7	B7_N0	PIN_C19	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[2]	Output	PIN_E21	6	B6_N0	PIN_E21	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[1]	Output	PIN_E22	6	B6_N0	PIN_E22	3.3-V LVTTL		8mA (default)	2 (default)				
out HEX3[0]	Output	PIN_F21	6	B6_N0	PIN_F21	3.3-V LVTTL		8mA (default)	2 (default)				
in KEY[0]	Input	PIN_B8	7	B7_N0	PIN_B8	3.3-V LVTTL		8mA (default)					
out LEDR[9]	Output	PIN_B11	7	B7_N0	PIN_B11	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[8]	Output	PIN_A11	7	B7_N0	PIN_A11	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[7]	Output	PIN_D14	7	B7_N0	PIN_D14	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[6]	Output	PIN_E14	7	B7_N0	PIN_E14	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[5]	Output	PIN_C13	7	B7_N0	PIN_C13	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[4]	Output	PIN_D13	7	B7_N0	PIN_D13	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[3]	Output	PIN_B10	7	B7_N0	PIN_B10	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[2]	Output	PIN_A10	7	B7_N0	PIN_A10	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[1]	Output	PIN_A9	7	B7_N0	PIN_A9	3.3-V LVTTL		8mA (default)	2 (default)				
out LEDR[0]	Output	PIN_A8	7	B7_N0	PIN_A8	3.3-V LVTTL		8mA (default)	2 (default)				
in mosi	Input	PIN_AB8	3	B3_N0	PIN_AB8	3.3-V LVTTL		8mA (default)					on
in sclk	Input	PIN_AB5	3	B3_N0	PIN_AB5	3.3-V LVTTL		8mA (default)					on
out speaker_out	Output	PIN_W10	3	B3_N0	PIN_W10	3.3-V LVTTL		8mA (default)	2 (default)				
in SW0	Input	PIN_C10	7	B7_N0	PIN_C10	3.3-V LVTTL		8mA (default)					
in SW1	Input	PIN_C11	7	B7_N0	PIN_C11	3.3-V LVTTL		8mA (default)					
in SW2	Input	PIN_D12	7	B7_N0	PIN_D12	3.3-V LVTTL		8mA (default)					
in SW3	Input	PIN_C12	7	B7_N0	PIN_C12	3.3-V LVTTL		8mA (default)					
in SW8	Input	PIN_B12	7	B7_N0	PIN_B12	3.3-V LVTTL		8mA (default)					
in SW9	Input	PIN_F15	7	B7_N0	PIN_F15	3.3-V LVTTL		8mA (default)					
<<new node>>													

Pins

קודים :Verilog:

שעון עצר :

הרכיב כולל לוגיקת סנכרון כפול (Synchronizer) ומסנן רעשים (Debouncer) למניעת קפיצות בשעון כתוצאה מהפרעות חשמליות.

```verilog
    input clk,          // 50 מחברת ה-50MHz
    input reset,        // ל מחוגי-SW9
    input key_reset,    // ל מחוגי-KEY0 (Active Low)
    input run_enable,   // ל מחוגי-is_active (מאות המאה)
    input pause,        // ל מחוגי-SW8
    output [7:0] hex0, hex1, hex2, hex3
);

    reg [25:0] clk_count = 0;
    reg [3:0] sec_low=0, sec_high=0, min_low=0, min_high=0;

    // === 1 שלב: Synchronizer (מניעת Metastability) ===
    reg sync1, sync2;
    always @(posedge clk) begin
        sync1 <= run_enable;
        sync2 <= sync1;
    end

    // === 2 שלב: Debouncer (מיזון רעשים וקפיצות מוניטין) ===
    // מונה 1,000,000 חזורי שמון (20-) מיליי-שניה) אל יציבות
    reg [19:0] db_count = 0;
    reg run_clean = 0;

    always @(posedge clk) begin
        if (sync2 != run_clean) begin
            if (db_count >= 1000000) begin
                run_clean <= sync2;
                db_count <= 0;
            end else begin
                db_count <= db_count + 1;
            end
        end else begin
            db_count <= 0;
        end
    end

    // === לוגיקת השעון המרכזית ===
    wire master_reset = reset || !key_reset; // איפוס מהמוניטין' או מהכפתור

    always @(posedge clk) begin
        if (master_reset) begin
            clk_count <= 0;
            sec_low <= 0; sec_high <= 0; min_low <= 0; min_high <= 0;
        end
        else if (run_clean && !pause) begin // מיחוש מאות הנקי והמחוברן
            if (clk_count >= 49999999) begin // מגיעים שניה אחת
                clk_count <= 0;
                if (sec_low == 9) begin
                    sec_low <= 0;
                    if (sec_high == 5) begin
                        sec_high <= 0;
                        if (min_low == 9) begin
                            min_low <= 0;
                            if (min_high == 5) min_high <= 0;
                            else min_high <= min_high + 1;
                        end else min_low <= min_low + 1;
                    end else sec_high <= sec_high + 1;
                end else sec_low <= sec_low + 1;
            end else clk_count <= clk_count + 1;
        end
    end

    // 7 פונקציית שמרה לתצוגה Segment
    function [7:0] decode (input [3:0] n);
        case (n)
            4'h0: decode = 8'hC0; 4'h1: decode = 8'hF9;
            4'h2: decode = 8'hA4; 4'h3: decode = 8'hB0;
            4'h4: decode = 8'h99; 4'h5: decode = 8'h92;
            4'h6: decode = 8'h82; 4'h7: decode = 8'hF8;
            4'h8: decode = 8'h80; 4'h9: decode = 8'h90;
            default: decode = 8'hFF;
        endcase
    endfunction

    assign hex0 = decode(sec_low);
    assign hex1 = decode(sec_high);
    assign hex2 = decode(min_low) & 8'h7F; // נקודה משרוניה בין דקות לשניות
    assign hex3 = decode(min_high);

endmodule
```

רכיב הSPI SLAVE :

קולט נתוני 10-ביט מהראספברי פיי

```
module spi_slave (
    input wire sclk,        // שעון התקשורת (מהראספברי)
    input wire cs,          // (מסמן מתי מתחילים ומסיימים) צ'יפ סלקט
    input wire mosi,        // (הביטים עצמם) קו המידע
    output reg [9:0] pitch_val // המספר שיוצא החוצה לסינתיסייזר
);

    // אוגר (זיכרון) זמני לשמירת הביטים בזמן שהם נכנסים
    reg [9:0] shift_reg;

    // 1. תהליך הקליטה (Shifting)
    // עולה, אנחנו דוחפים ביט אחד פנימה (sclk) בכל פעם שהשעון
    always @(posedge sclk) begin
        if (cs == 0) begin // עובדים רק כשה-CS למטה (Active Low)
            shift_reg <= {shift_reg[8:0], mosi};
        end
    end

    // 2. תהליך העדכון (Latching)
    // עולה חזרה למעלה (סוף השידור), אנחנו מעדכנים את היציאה רק כשה-CS
    always @(posedge cs) begin
        pitch_val <= shift_reg;
    end

endmodule
```

מחולל גל ריבועי : מחולל תדרים דיגיטלי הממיר את ערכי ה SPI-לצליל

```
module square_wave_gen (
    input clk,              // 50 שעון מערכת MHz
    input enable,           // מהדיטקטור-is_active מחובר ל
    input pause,            // 8 כניסה חדשה לסוויץ'
    input [9:0] pitch_val,  // (0-1023) הערך שמגיע מהפאי
    output reg wave_out     // היציאה לרמקול
);

    integer counter;

    // חישוב הגבול לספירה לפי התדר המבוקש
    // ככל ש-pitch_val גדול יותר, limit יותר נבוה יותר קטן יותר
    wire [31:0] limit = 200000 - (pitch_val * 150);

    initial begin
        counter = 0;
        wave_out = 0;
    end

    always @(posedge clk) begin
        // התנאי להפעלת הצליל: יש אישור נגינה, אין פאוז ויש ערך תדר
        if (enable == 1 && pause == 0 && pitch_val > 0) begin
            if (counter >= limit) begin
                counter <= 0;
                wave_out <= ~wave_out; // הפיכת האות ליצירת גל ריבועי
            end else begin
                counter <= counter + 1;
            end
        end
        else begin
            // אם המנגינה נעצרה או שיש פאוז - שקט מוחלט ואיפוס
            wave_out <= 0;
            counter <= 0;
        end
    end

endmodule
```

Led dancer :

```
module led_dancer (
    input clk,
    input [9:0] pitch_val,
    input [3:0] brightness, // חבר לסוויצ'ים SW[3:0]
    output [9:0] ledr
);
    // לוגיקת PWM לעוצמת האור
    reg [3:0] pwm_count = 0;
    always @(posedge clk) pwm_count <= pwm_count + 1;
    wire light_on = (pwm_count < brightness);

    // לוגיקת הריקוד
    reg [9:0] dance_pattern;
    always @(*) begin
        dance_pattern = 10'b0;
        if (pitch_val > 150) dance_pattern[0] = 1;
        if (pitch_val > 250) dance_pattern[1] = 1;
        if (pitch_val > 300) dance_pattern[2] = 1;
        if (pitch_val > 350) dance_pattern[3] = 1;
        if (pitch_val > 400) dance_pattern[4] = 1;
        if (pitch_val > 450) dance_pattern[5] = 1;
        if (pitch_val > 500) dance_pattern[6] = 1;
        if (pitch_val > 550) dance_pattern[7] = 1;
        if (pitch_val > 600) dance_pattern[8] = 1;
        if (pitch_val > 650) dance_pattern[9] = 1;
    end

    // שילוב העוצמה עם הריקוד
    assign ledr = light_on ? dance_pattern : 10'b0;
endmodule
```

מערכת זיהוי הפעלת הנגינה:

```
module activity_detector (
    input clk,          // 50MHz
    input cs,           // Chip Select 5 מהראספברי פאי
    output is_active    // יוצא ל-run_enable של השעון
);

    // 1. סנכרון האות (למניעת רעשים ומטא-סטביליות)
    reg cs_sync_0, cs_sync_1;
    always @(posedge clk) begin
        cs_sync_0 <= cs;
        cs_sync_1 <= cs_sync_0;
    end

    // 2. זיהוי ירידה (Falling Edge)
    reg cs_prev;
    always @(posedge clk) cs_prev <= cs_sync_1;
    wire edge_detected = (cs_prev == 1 && cs_sync_1 == 0);

    // 3. טיימר השהייה (Keep-alive)
    // 25,000,000 = חצי שנייה בדיוק ב-50MHz מהדורים
    reg [24:0] timeout_timer = 0;

    always @(posedge clk) begin
        if (edge_detected) begin
            timeout_timer <= 25000000; // טעינה מחדש בכל פעם שיש תקשורת
        end else if (timeout_timer > 0) begin
            timeout_timer <= timeout_timer - 1;
        end
    end

    // 4. יציאה: השעון רץ רק כשהטיימר פועל
    assign is_active = (timeout_timer > 0);

endmodule
```

:Python code : RPI5

התוכנה שנכתבה בשפת Python רצה על גבי ה Raspberry Pi 5-ותפקידה לנהל את המלודיה ולשלוט בחומרה בזמן אמת.

ספריות בשימוש :

spidev ספריית חומרה המשמשת לתקשורת SPI מהירה. הספרייה מאפשרת שליטה ישירה בבקר ה SPI-המובנה של הראספברי פיי לסנכרון נתונים מדויק

RPi.GPIO ספרייה לניהול פיני ה .GPIO-בפרויקט זה היא משמשת לקריאת אות ה Reset-מלוח ה) FPGA-דרך ,(GPIO 17 מה שמאפשר סנכרון בין לחיצה פיזית על הלוח לבין אתחול המנגינה בתוכנה.

time ספרייה לניהול תזמון, המשמשת לקביעת משך הנגינה של כל תו (Duration)והשהיות לצורך הפרדת צלילים(Staccato).

לוגיקה מרכזית וחישוב תדרים:

pitch_val = (200,000 - (25,000,000 / frequency)) / 150

פרוטוקול העברת הנתונים:

הבייט הראשון מכיל את ה-MSB (הביטים הגבוהים).

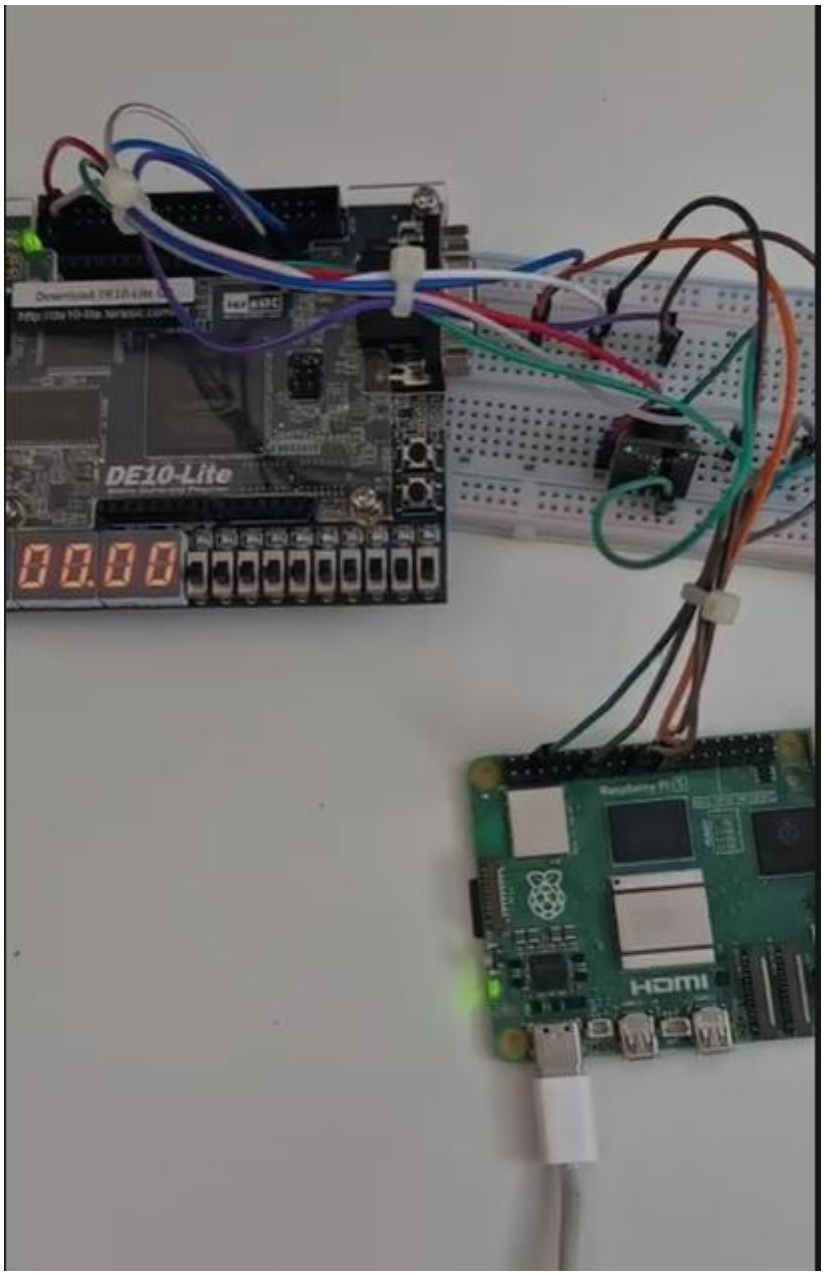
הבייט השני מכיל את ה-LSB (הביטים הנמוכים).

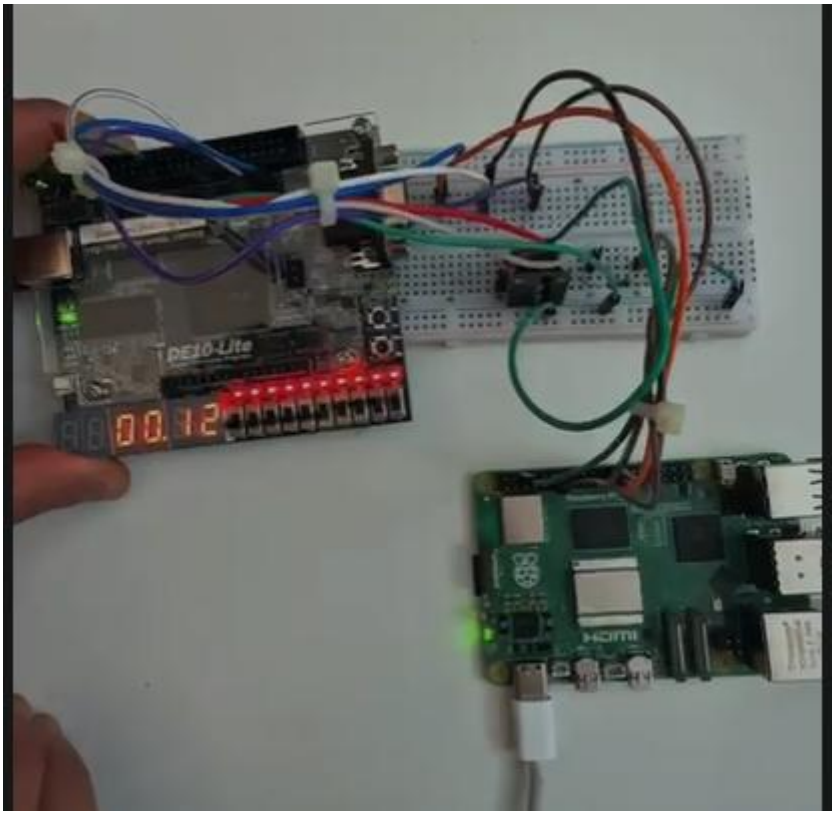
הפרדה זו מאפשרת שליחת ערך מדויק של 10-ביט המייצג את גובה הצליל ואת מצב הלדים.

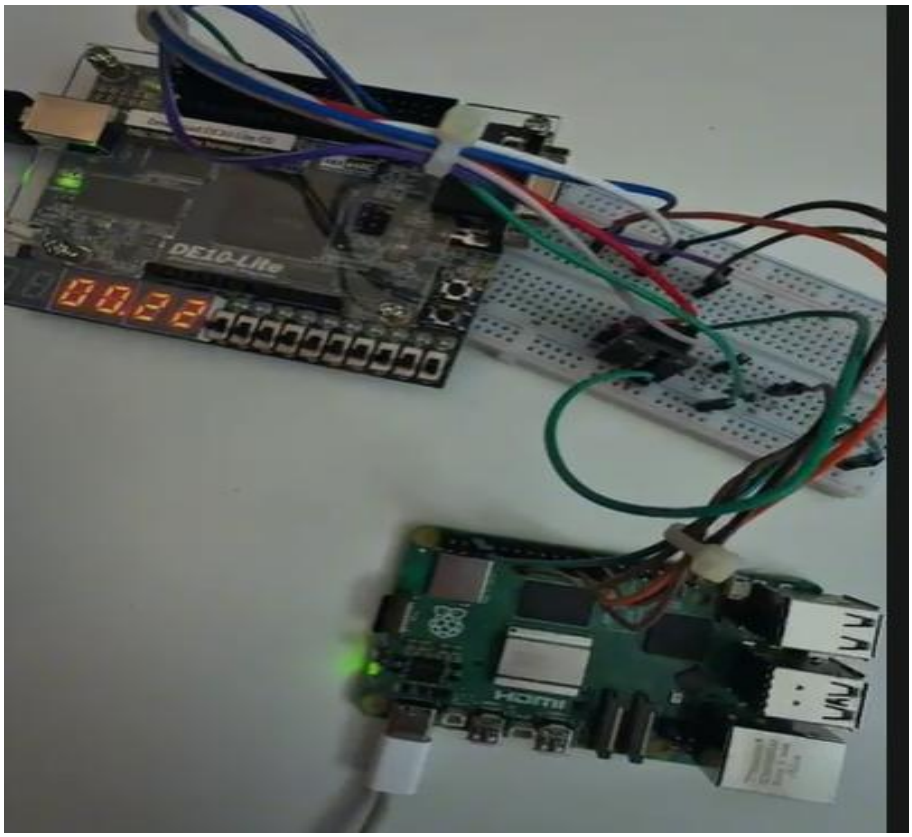
C: > Users > dilen > Desktop > star_wars.py > ...

```python
import spidev
import time
import RPi.GPIO as GPIO

# הגדרת ה-SPI (חומרה)
spi = spidev.SpiDev()
spi.open(0, 0)
spi.max_speed_hz = 1000000

# הגדרת כפתור הריסט (KEY0 ב-FPGA)
RESTART_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(RESTART_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)

def send_pitch(frequency):
    if frequency == 0:
        pitch_val = 0
    else:
        # שלכם Verilog-נוסחת ההמרה המדויקת מה
        limit = 25000000 / frequency
        pitch_val = int((200000 - limit) / 150)

    pitch_val = max(0, min(1023, pitch_val)) # וידוי טווח 10 ביט
    high_byte = (pitch_val >> 8) & 0x03
    low_byte = pitch_val & 0xFF
    spi.xfer2([high_byte, low_byte])

# תדרים למנגינת Star Wars (Imperial March)
NOTES = {
    'A3': 220.00, 'F3': 174.61, 'C4': 261.63, 'A4': 440.00,
    'E4H': 659.25, 'F4H': 698.46, 'GS3': 207.65, 'REST': 0
}

# מבנה המנגינה (תווים וזמנים)
melody = [
    ('A3', 0.5), ('A3', 0.5), ('A3', 0.5), ('F3', 0.35), ('C4', 0.15),
    ('A3', 0.5), ('F3', 0.35), ('C4', 0.15), ('A3', 0.65),
    ('REST', 0.1),
    ('A4', 0.5), ('A4', 0.5), ('A4', 0.5), ('F4H', 0.35), ('C4', 0.15),
    ('GS3', 0.5), ('F3', 0.35), ('C4', 0.15), ('A3', 0.65),
    ('REST', 0.2)
]

try:
    print("🎵 Playing Star Wars Imperial March (SPI Mode)... 🎵")
    print("Presented by Dylan Gershon & Shahar Katz") #

    while True:
        for note, duration in melody:
            # בדיקת איפוס מה-FPGA
            if GPIO.input(RESTART_PIN) == GPIO.LOW:
                print("Restart signal detected!")
                break

            send_pitch(NOTES[note])
            time.sleep(duration)

            # הפרדה קטנה בין תווים לסאונד נקי (Staccato)
            send_pitch(0)
            time.sleep(0.02)

        print("Long live the Empire! Repeating...")

except KeyboardInterrupt:
    print("\nStopping...")
    send_pitch(0)
    spi.close()
    GPIO.cleanup()
```

צילומים של המערכת בפעולה :







הסבר על החיבור למטריצה :

תיאום רמות מתח (Logic Levels): מכיוון שגם ה Raspberry Pi 5-וגם ה-FPGA DE10-Liteעובדים ברמת לוגיקה של 3.3 {V}, בוצע חיבור ישיר ביניהם ללא צורך בממירי רמות מתח (Level Shifters), תוך הקפדה על הגדרת הפינים ב Quartus-כ.3.3-V LVTTL-

מבנה החיבור במטריצה :הקישור בוצע באמצעות 4 קווי תקשורת עיקריים (MOSI, SCLK, CS, Reset)וחיבור אדמה משותף (Common Ground) בין המכשירים כדי להבטיח ייחוס מתחים תקין ומניעת רעשים בסיגנל.

מניעת רעשים חשמליים :בשל השימוש במטריצה וחוטים ארוכים, הוטמעו בקוד ה Verilog-רכיבי סנכרון (Synchronizers) לטיפול בתופעות של Metastabilityשנובעות מהבדלי השעונים בין שתי המערכות